

Chapter 1

Introduction: TypeScript, Deno and Celbridge

Object-oriented TypeScript, test first

```
Hello, world!
```

Today

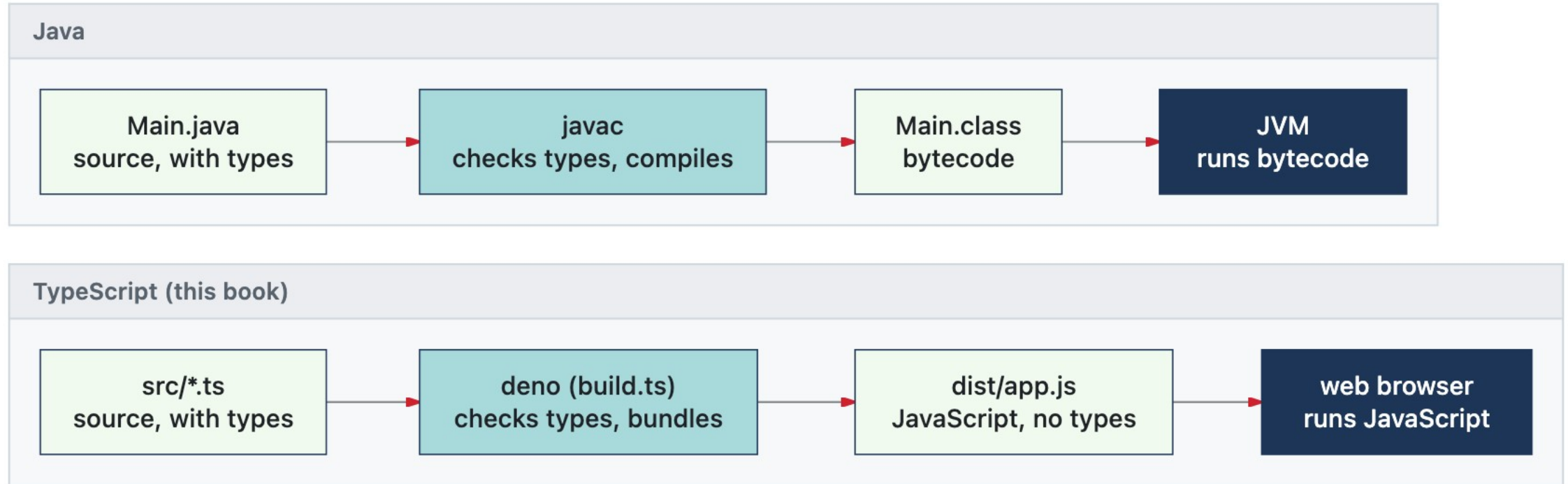
- what TypeScript is - and how it compares with Java
- opening a project: it builds, tests and watches by itself
- **Hello, world!** - then the time of day, then functions
- **checking**: doc comment examples, tests, **test-driven development** (TDD)
- arrays, lists on the page, and JSON data

Seven small projects - each a little bigger than the last

What is TypeScript?

- browsers run one language: **JavaScript** - no types
- **TypeScript** = JavaScript + types
- types are **checked**, then **removed**
- the browser never sees a type
- classes, interfaces, `extends`, `private` ... as in Java (from Chapter 2)

Java vs TypeScript



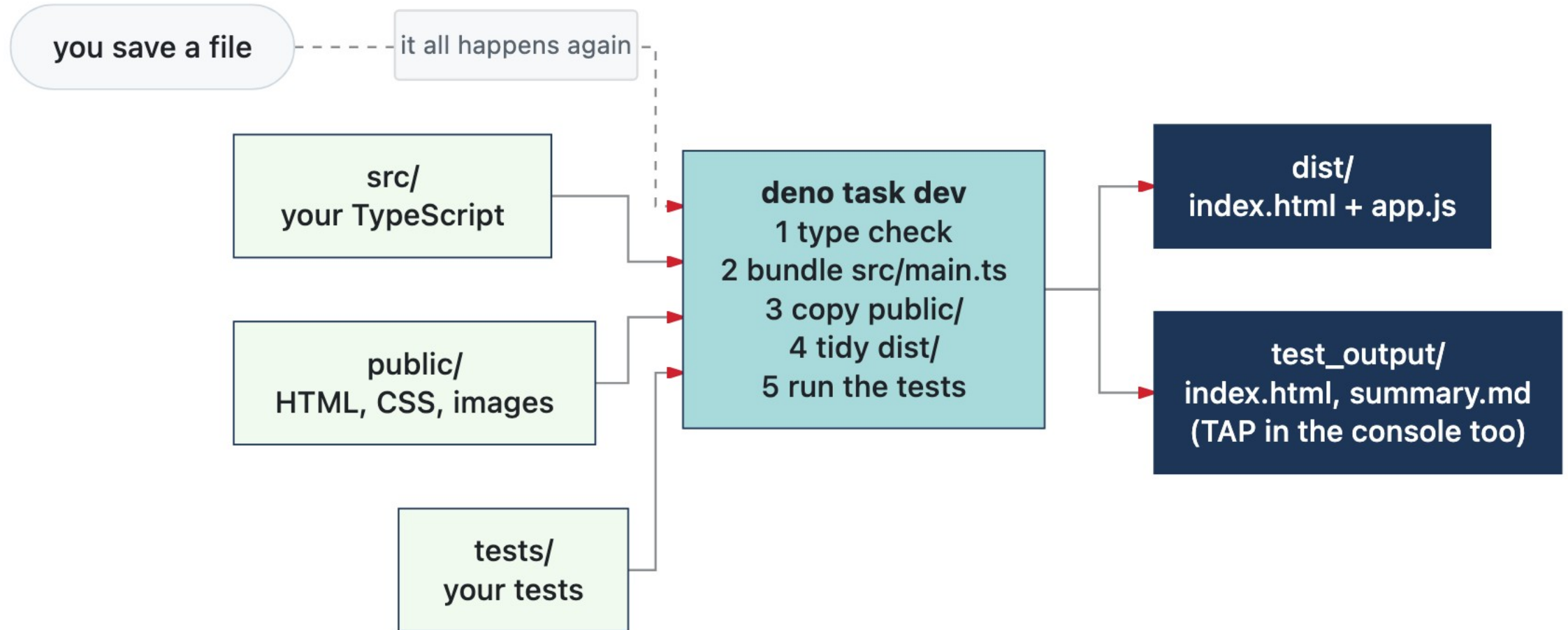
Types are checked before the program runs - the browser never sees a type

The tools

- **Deno** - type checker, bundler, test runner, linter: all built in
- **Celbridge** - open a project folder, and three things open:
 - the **console** - runs `deno task dev`
 - `dist/index.html` - your page, as a preview
 - `README.md`

No web server needed

Every time you save



Press **refresh** on the preview when it lights up

The reload icon indicates it's time to REFRESH



The image shows a development environment with a code editor on the left and a web browser on the right. The code editor has a tab for 'main.ts' containing TypeScript code. The browser has a tab for 'index.html' and displays the text 'Hello, Matt !'. A red callout bubble points from the browser's reload icon to the text 'Source changed -> new JS -> reload HTML'. Another red circle highlights the string 'Hello, Matt !' in the code.

```
TS main.ts ×  
1 // The whole program. Find the element on the page  
  whose id is "output", and change its text.  
2 //  
  
5 const output = document.querySelector("#output");  
6 if (output !== null) {  
7   output.textContent = "Hello, Matt !";  
8 }  
9
```

Source changed -> new JS -> reload HTML

index.html ×

Hello, Matt !

Edit these, look at those

Edit	Look at (never edit)
<code>src/</code> - your TypeScript	<code>dist/</code> - the built page
<code>tests/</code> - your tests	<code>test_output/</code> - the test report
<code>public/</code> - HTML, CSS, images	

Project 1: Hello, world!

```
const output = document.querySelector("#output");  
if (output !== null) {  
  output.textContent = "Hello, world!";  
}
```

```
<h1 id="output">(if you can read this, ...)</h1>  
<script src="app.js"></script>
```

- `querySelector` finds an element - or gives `null`
- TypeScript **makes** you check for `null`

Exercise 1.1 - Hello, you

```
const NAME = "Ada";

const output = document.querySelector("#output");
if (output !== null) {
  output.textContent = `Hello, ${NAME}!`;
}
```

- `const` - can never change (Java `final`); type is **inferred**
- backticks + `${...}` - a **template literal**

Project 2: What's the time?

```
const now = new Date();  
const hour: number = now.getHours();
```

- types come **after** the name
- one `number` type: `7 / 2` is `3.5`
- `string`, `boolean`, `string[]`, `void`

Exercise 1.2 - Good morning

```
let message = "Good day!";  
if (hour < 12) {  
  message = "Good morning!";  
}
```

- `let` - a variable that can change; `const` otherwise; never `var`
- compare with `===` and `!==` - never `==` (`0 == ""` is `true` !)
- `"cat" === "cat"` - no `.equals()`

Exercise 1.3 - A function of its own

```
// greeting.ts
const MIDDAY = 12;

export function greeting(hour: number): string {
  if (hour < MIDDAY) {
    return "Good morning";
  }
  return "Good day";
}

// main.ts
import { greeting } from "../greeting.ts";
```

Functions need no class · `export` / `import` · paths end `.ts`

Why test?

- does `greeting` get midday right? midnight?
- checking by hand: wait until the afternoon ... every time you change the code
- a **test** runs your code and checks the answer - instantly, every save
- `greeting(hour)` takes the hour as a parameter, so a test can try **any** hour
- no page code in `greeting.ts` → Deno can test it without a browser

A first check: an example

src/greeting.ts

```
/**
 * "Good morning" before midday, "Good day" from midday on.
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(greeting(9), "Good morning");
 * assertEquals(greeting(15), "Good day");
 * ```
 */
export function greeting(hour: number): string {
```

- Deno runs the example as a test, every save
- `assertEquals(actual, expected)` - actual first

The first result

```
TAP version 14
# src/greeting.ts
ok 1 - example in the doc comment (lines 11-17)
1..1
```

Tests: 0 failed, 1 passed, 0 skipped, 0 type errors, ...

All tests passing
ch01_project03_first_tests - 10/4/2026, 5:28:44 PM

1 passed	0 failed	0 skipped	0 type errors	0 lint warnings
-------------	-------------	--------------	------------------	--------------------

All tests
src/greeting.ts
✓ example in the doc comment (lines 11-17)

▶ TAP output

Made by tools/test_report.ts - Markdown version: summary.md

A comment is not a check: `greeting(9); // "Good evening"` passes

When a check fails

Change `MIDDAY` to `8` and save:

```
not ok 1 - example in the doc comment (lines 11-17)
  message: |-
    AssertionError: Values are not equal.

      [Diff] Actual / Expected

      - Good day
      + Good morning
      at src/greeting.ts (example at lines 11-17)
```

– what the code **actually** gave · + what was **expected**

Examples are not enough

What about 11 o'clock? Midday? Midnight?

Put them all in the doc comment and:

- the example is no longer short and clear
- an example has **no name** saying what it checks
- the first failing `assertEquals` **stops** the rest

So: **test files** - `tests/*.test.ts`

A test file

```
import { assertEquals } from "@std/assert";

Deno.test("one plus one is two", () => {
  assertEquals(1 + 1, 2);
});
```

- a **name** (a sentence) and the **code** that checks it
- `() => { ... }` - "the test's code" (Chapter 3)
- each test runs on its own: one failure does not stop the others

Testing the greeting

```
Deno.test("11 o'clock is still morning", () => {  
  assertEquals(greeting(11), "Good morning");  
});  
  
Deno.test("midday (hour 12) is day", () => {  
  assertEquals(greeting(12), "Good day");  
});
```

A test file imports what it tests: `../src/greeting.ts`

Test the edges - `<=` instead of `<` only shows up at 12

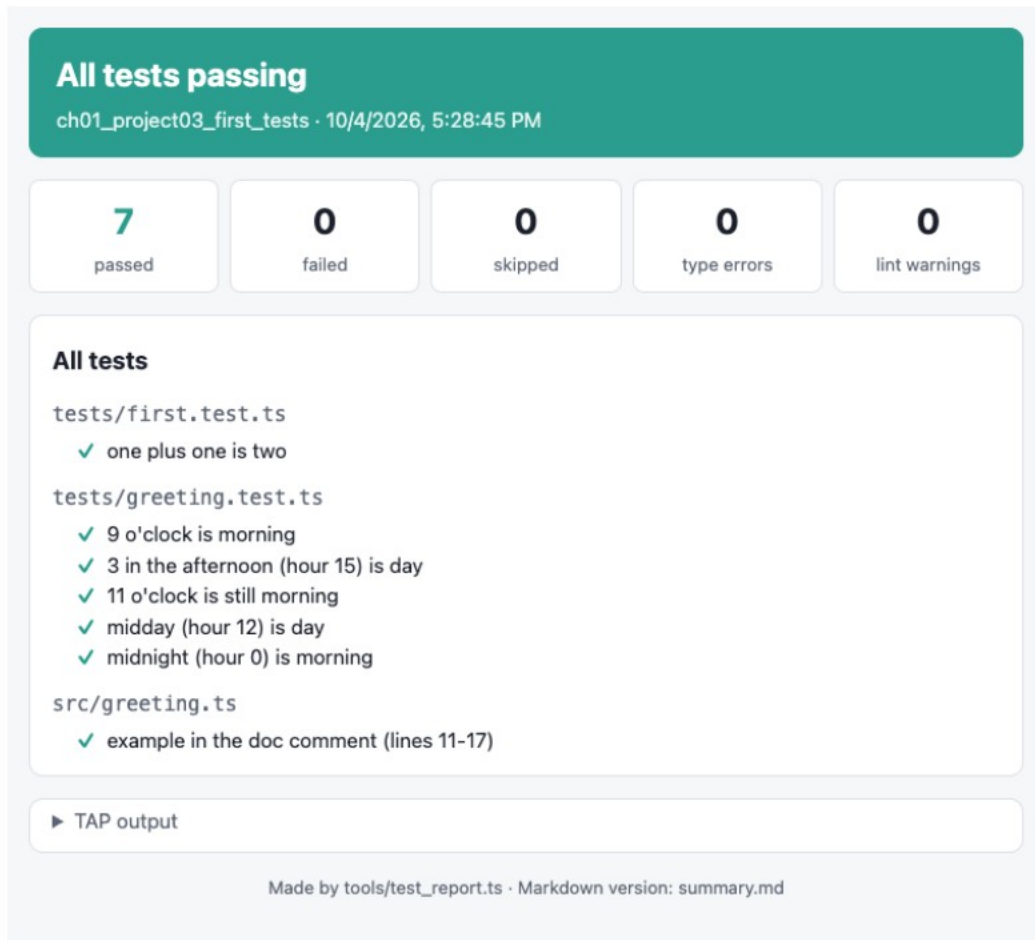
The results

```
TAP version 14
# tests/first.test.ts
ok 1 - one plus one is two
# tests/greeting.test.ts
ok 2 - 9 o'clock is morning
...
# src/greeting.ts
ok 7 - example in the doc comment (lines 11-17)
1..7

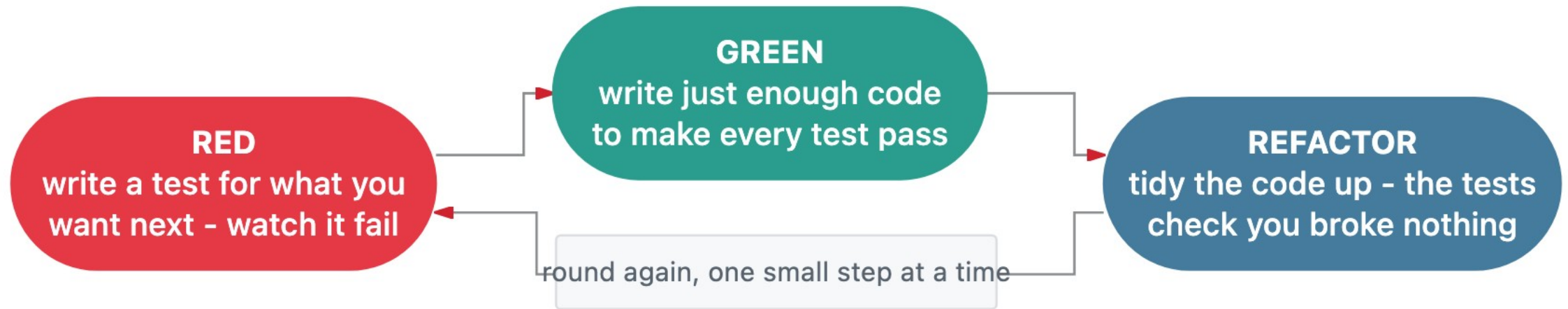
Tests: 0 failed, 7 passed, 0 skipped, 0 type errors, 0 lint warnings
```

Examples show typical use · **tests** check the edges, by name

The test report



Test-driven development



Write the test **first** - then just enough code to pass

Project 4: Age groups

- cage categories:
 - under 12: child
 - 13-19: teenager
 - 20+: adult
 - below 0 or over 150: invalid
- **What about 12?** Tests need exact answers - so they find the gaps
- step 1: a test for "child" → **red** (no function yet)
- step 2: `return "child";` → **green** (yes, really)

Red again

- step 3: a test for "teenager" → **red**
- the diff: `-` actual is `child`, `+` expected is `teenager`
- step 4: add an `if` → **green**
- ... one group at a time, then the edges

The report after step 3

1 test failing
ch01_project04_age_groups · 10/3/2026, 2:27:21 PM

1 passed	1 failed	0 skipped	0 type errors	0 lint warnings
-------------	-------------	--------------	------------------	--------------------

Failing tests
✗ a 15 year old is a teenager
tests/age_group.test.ts:8

AssertionError: Values are not equal.

[Diff] Actual / Expected

- child
+ teenager
at tests/age_group.test.ts:9:3

All tests
tests/age_group.test.ts
✓ a 5 year old is a child
✗ a 15 year old is a teenager

▶ TAP output

Made by tools/test_report.ts · Markdown version: summary.md

Refactor

```
const OLDEST_CHILD = 12;
const OLDEST_TEENAGER = 19;
const OLDEST_POSSIBLE = 150;

export function ageGroup(age: number): string {
  if (age < 0 || age > OLDEST_POSSIBLE) {
    return "invalid";
  }
  if (age <= OLDEST_CHILD) {
    return "child";
  }
  if (age <= OLDEST_TEENAGER) {
    return "teenager";
  }
  return "adult";
}
```

Still green → tidy-up safe · then add an `@example` doc comment

Project 5: A bulleted list

```
const PLANETS: string[] = ["Mercury", "Venus", "Earth", "Mars"];

for (const planet of PLANETS) {
  const item = document.createElement("li");
  item.textContent = planet;
  list.appendChild(item);
}
```

- `for ... of` - not `for ... in`
- `.length` - no brackets

Exercises 1.4 and 1.5 - data in a file

```
// planets.ts  
export const PLANETS: string[] = ["Mercury", "Venus"];
```

```
["Mercury", "Venus", "Earth", "Mars"]
```

```
import planets from "./planets.json" with { type: "json" };  
const PLANETS: string[] = planets;
```

The build reads and **parses** the JSON → no server needed

Project 7: Putting it together

- `partOfDay(hour)` - morning, afternoon, evening; `greeting(name, hour)`
- a list of facts from `facts.ts`, and tests for it all
- nothing new - just everything at once

Good afternoon, student!

It is 2:27:19 PM.

5 things about TypeScript

- TypeScript is JavaScript with types added.
- The types are checked, then removed - browsers run plain JavaScript.
- There is one type for numbers, `number` - no `int`, `long`, `float` or `double`.
- `let` declares a variable that can change; `const` declares one that cannot.
- Strings are compared with `===`, not with an `equals()` method.

Where problems show up

Problem	Where
type error	console + report
failing test or example	console (TAP) + report
lint warning	console + report
wrong <code>id</code> at run time	browser developer tools

Summary

- TypeScript = JavaScript + types; checked, then removed
- save → type check, build, test, report
- `const` / `let` ; types after names; `${}` ; `===`
- functions in any file; `export` / `import`
- `@example` in a doc comment: checked documentation
- test files: the edges, by name; `assertEquals(actual, expected)`
- **red** → **green** → **refactor**, in small steps
- arrays, `for ... of` , JSON data

Challenges

1. **Make it yours** - name, a sixth fact, colours
2. **Good night** - hours 22-4, test first
3. **Break it on purpose** - four mistakes; where does each show?
4. **Seniors** - a fifth age group, test first
5. **Planets with facts** - objects in JSON
6. **Day and night colours** - `themeFor(hour)` , tested